Noise & Restoration

Week 4 — Session 2: Edge-preserving restoration & measuring quality

Practical Image Processing

Lecture 4.2 ▪ 60–70 min

Recap of Session 1

- Noise **type** decides the cure: Gaussian/Poisson (grain) vs salt-and-pepper (specks).
- **Averaging** cuts Gaussian noise by $\sqrt{N}$ — but blurs edges.
- **Median** deletes impulses and keeps step edges — but can't help grain much.
- Today: keep edges *and* kill grain, handle hard cases, and *measure* the result.

The open problem

Plain averaging forces a choice: smooth flats *or* keep edges. Can we have both?

Roadmap

Edge-preserving filters

Adaptive & order-statistic filters

Temporal denoising

Measuring restoration

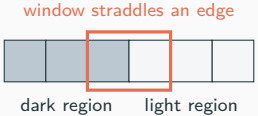
Deblurring & practice

Wrap-up & deliverable

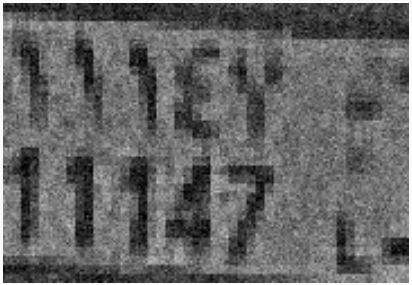
Edge-preserving filters

The problem with plain averaging

- A box/Gaussian kernel averages *every* neighbour — including the ones on the *other* side of an edge.
- So a sharp digit stroke on a light background gets a gray halo: the edge is blurred.
- Fix: average a neighbour *only if* it really belongs to the same region.



The bilateral filter



noisy



- Weight each neighbour by *two* things: how **close** it is (space) *and* how **similar** its intensity is (range).
- Neighbours across an edge are far in *intensity* $\rightarrow$ they get almost zero weight.
- Result: flats are smoothed, edges are preserved.

✔ Good practice

For DocScan: bilateral before thresholding keeps digit edges crisp while killing sensor grain.

Bilateral filter — the math

Math

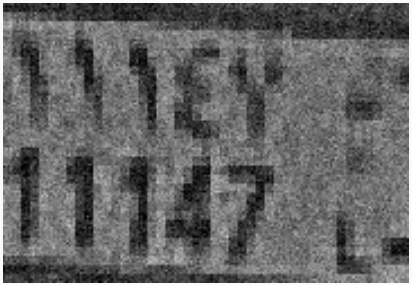
$$g(p) = \frac{1}{W_p} \sum_{q \in \Omega} \underbrace{G_{\sigma_s}(\|p - q\|)}_{\text{spatial}} \underbrace{G_{\sigma_r}(|I(p) - I(q)|)}_{\text{range}} I(q)$$

- σ_s: how far in *space* we look; σ_r: how different in *intensity* we still average.
- Small $\sigma_r \rightarrow$ only near-identical pixels count $\rightarrow$ strong edge preservation.
- Large $\sigma_r \rightarrow$ behaves like an ordinary Gaussian blur (edges blur again).

Note

Two knobs, two jobs: σ_s controls *reach*, σ_r controls *edge sensitivity*.

Non-local means (NLM)



noisy

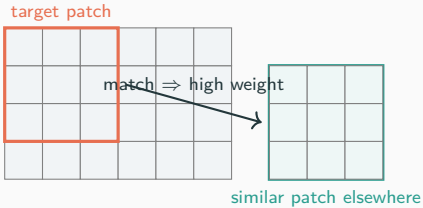


- Idea: average pixels whose surrounding *patch* looks similar — even if they are far apart.
- An image repeats itself (many similar textures, strokes, flats); NLM exploits that redundancy.
- Best quality on Gaussian noise, at a real speed cost.

Key idea

Bilateral compares single pixels; NLM compares whole neighbourhood *patterns* — more robust.

NLM: comparing patches, not pixels



- Weight $\propto \exp\big(-\|P(p)-P(q)\|^2/h^2\big)$ where $P(\cdot)$ is a small patch around a pixel.
- Larger h = stronger denoising = more risk of washing out fine detail.

Bilateral vs NLM — how to choose

	Bilateral	**Non-local means**
Compares	single pixel intensity	whole patch pattern
Search area	local window	(semi-)global
Speed	fast	slow
Quality on grain	good	best
Best for	real-time / edges	offline / max quality

✔ Good practice

Reach for bilateral when you need speed and crisp edges; reach for NLM when quality is worth the wait.

Edge-preserving filters in code

```python
import cv2
# bilateral: d=9 window, sigmaColor=60 (range), sigmaSpace=60 (spatial)
bilateral = cv2.bilateralFilter(noisy, 9, 60, 60)

# non-local means: h=15 strength, templateWindow=7, searchWindow=21
nlm = cv2.fastNlMeansDenoising(noisy, None, 15, 7, 21)
```

Note

Bigger `sigmaColor`/`h` = stronger denoising but more lost detail. Tune on *your* hard images, then measure.

Adaptive & order-statistic filters

The order-statistic family

- These filters **rank** the window's values and pick one — no averaging at all.
- **Median** = middle rank; **min** filter = rank 1 (removes salt); **max** filter = top rank (removes pepper).
- Because they return a *real* pixel value, they never invent gray blends across edges.

Math

Sort the window $\{x_{(1)} \leq \cdots \leq x_{(N)}\}$: $\quad \min = x_{(1)}, \quad \text{median} = x_{((N+1)/2)}, \quad \max = x_{(N)}.$

Adaptive median filter

- Plain median with a big window blurs; with a small window it misses dense impulse noise.
- **Adaptive median**: start small; *grow* the window only where the centre still looks like an impulse.
- Cleans heavy salt-and-pepper while leaving clean regions almost untouched.

✓ Good practice

For very noisy meter scans, an adaptive median can save digits a fixed median would smear.

Worked: adaptive median in action

A pixel sits in heavy pepper; the 3×3 window is mostly **0**s:

0	0	131
0	0	0
128,0,0		

1. 3×3 median $= 0$ — but 0 is itself an impulse, so *don't* trust it: grow the window.
2. At 5×5 enough true pixels ($\approx$ 128–131) appear; the median lands near **130**.
3. Output **130** — a real gray, not a pepper dot. Fixed median would have output 0.

Temporal denoising

Temporal denoising: average many frames

- If you have several shots of the *same* static scene, average them: noise cancels, signal stays.
- N frames reduce Gaussian noise by $\sqrt{N}$ — the cheapest, highest-quality denoiser, with *zero* blur.
- For a meter you can photograph twice, this beats any single-image filter.

✔ Good practice

When you control capture, averaging a burst is the best denoising you can do — no detail lost.

Worked: how many frames?

Math

$$\sigma_{\text{out}} = \frac{\sigma}{\sqrt{N}} \;\Rightarrow\; N = \left(\frac{\sigma}{\sigma_{\text{target}}}\right)^2.$$

- Grain has $\sigma = 12$; you want it down to $\sigma_{\text{target}} = 4$: $N = (12/4)^2 = 9$ frames.
- Want $\sigma = 2$ instead? $N = (12/2)^2 = 36$ frames — quadratic cost for each halving.
- Unlike spatial filters, this costs *no* sharpness — only capture time and a steady hand.

✕ Pitfall

The scene must be *static* and aligned. Any motion between frames turns averaging into motion blur.

Measuring restoration

Restoration vs enhancement

- **Enhancement** (Wk2) makes an image *look* better (subjective, no ground truth needed).
- **Restoration** tries to *recover* the original by modelling the degradation (objective).
- Because restoration has a target, we can put a *number* on success — that is what PSNR/SSIM do.

Key idea

If you have a clean reference (or a synthetic-noise experiment), *measure* — never just eyeball.

PSNR — peak signal-to-noise ratio

Math

$$\mathrm{MSE} = \frac{1}{MN}\sum_{x,y}\big(f(x,y) - \hat{f}(x,y)\big)^2, \qquad \mathrm{PSNR} = 10\log_{10}\frac{255^2}{\mathrm{MSE}}\ \mathrm{dB}.$$

- Compares your result $\hat{f}$ to a clean reference f; higher dB = closer.
- Rough guide: > 40 dB excellent, 30–40 good, < 30 visible degradation.
- Simple and standard — but it only counts *pixel* errors, not whether structure survived.

Worked: compute a PSNR

- Suppose after denoising the mean-squared error vs the clean image is $\mathrm{MSE} = 25$.
- $\mathrm{PSNR} = 10 \log_{10} \dfrac{255^2}{25} = 10 \log_{10}(2601) \approx 10 \times 3.415 = \mathbf{34.1}$ dB.
- Halve the error to $\mathrm{MSE} = 12.5$: PSNR rises about 3 dB to ≈ 37.1 dB (every $\times 2$ in MSE ≈ 3 dB).

Note

Because it is logarithmic, small dB gains matter: $+3$ dB means you *halved* the squared error.

SSIM — structural similarity

- **SSIM** compares *structure* (local luminance, contrast, correlation), not raw pixel error.
- Ranges 0 to 1; 1 = identical. Matches human judgement far better than PSNR.
- A denoiser can win on PSNR yet lose on SSIM if it smears the very edges you care about.

✔ Good practice

Report *both*: PSNR for the error magnitude, SSIM for whether the digits' structure survived.

The denoising dilemma

- Every denoiser trades **noise removal** against **detail preservation**.
- Remove too little → noise breaks the next stage; too much → you erase the digits.
- There is no free lunch: choose the operating point for *your* goal (readable digits).

Key idea

Denoise for the downstream task, measured on hard images — not for the prettiest flat wall.

Deblurring & practice

Deblurring & the PSF

- Blur = convolution with a **point-spread function** (PSF). Deblur = deconvolution.
- If you *know* the PSF (motion length, defocus radius), you can partly invert it.
- Real methods regularize: **Wiener** (frequency-domain, noise-aware), **Richardson–Lucy** (iterative).

Math

Degradation model: $g = h * f + n$. Naive inverse $\hat{f} = g/H$ divides by the PSF spectrum — dangerous where $H \approx 0$.

Why naive deconvolution explodes

- Where the PSF spectrum H is near zero, dividing by it *amplifies* any noise there to huge values.
- The result is dominated by amplified grain and ringing — worse than the blurry input.
- Regularization (Wiener) damps those frequencies using the noise-to-signal ratio.

✕ Pitfall

Perfect deblurring is impossible when information is truly lost. A steady, well-lit shot beats any deconvolution.

Where denoising sits in the pipeline



- Denoise *early* — before enhancement (which would amplify the noise) and before thresholding (which noise would corrupt).
- But not *too* aggressively: over-denoising erases the digits you need.

Deep learning denoisers (preview)

- Modern denoisers are CNNs trained on (noisy, clean) pairs — they beat classical methods on benchmarks.
- But they need data, compute, and can *hallucinate* detail that isn't there.
- For DocScan, classical (median/bilateral/NLM) is usually enough and fully interpretable.

✕ Pitfall

A hallucinated digit is worse than a noisy one — you cannot trust a reading you cannot audit.

In-class exercise (10 min)

Design & measure a denoising experiment

1. You must keep digit edges razor-sharp while removing grain — which filter, and which two knobs do you tune?
2. After denoising, $\mathrm{MSE} = 6.5$ vs the clean image. Estimate the PSNR in dB.
3. Your method scores high PSNR but low SSIM. What likely went wrong?
4. You can capture 4 or 16 burst frames. How much does each cut the noise σ?

Debrief

- (1) **Bilateral** (or NLM): tune σ_s (reach) and σ_r (edge sensitivity); keep σ_r small.
- (2) $\mathrm{PSNR} = 10 \log_{10}(255^2/6.5) \approx$ **40** dB — excellent.
- (3) It over-smoothed: pixel error is low but *structure* (edges) was washed out — SSIM caught it.
- (4) 4 frames $\to \sigma/2$; 16 frames $\to \sigma/4$. Noise falls as $\sqrt{N}$.

Key idea

Match the filter to the noise, then *prove* it helped with a number on hard images.

Wrap-up & deliverable

What you should now be able to do

- Use edge-preserving filters (bilateral, NLM) and explain *why* they keep edges.
- Reach for adaptive/order-statistic filters on heavy impulse noise, and temporal averaging when you control capture.
- Measure restoration with PSNR *and* SSIM instead of eyeballing.
- Reason about deblurring's limits and place denoising correctly in the pipeline.

This week's deliverable

Due before Week 5 (see `project/`)

1. Add Gaussian and salt-pepper noise to a clean meter image (known noise = ground truth).
2. Denoise each with 3 filters (e.g. median, bilateral, NLM); compute **PSNR** *and* **SSIM** vs the clean original; tabulate.
3. Show a case where the "wrong" filter fails (Gaussian on salt-pepper), and one where PSNR and SSIM *disagree*.
4. DocScan: denoise a real noisy meter photo so its Otsu/adaptive binary is clean and the digits stay sharp.

Keep the edges, kill the noise.

Bilateral/NLM for grain, adaptive median for specks — and measure with PSNR + SSIM.